name in this example. `Optional<T>` is returned to safely handle the case where no matching record exists [25].

2.7. Seed reference / lookup data

The lookup tables within the initial schema were populated with predefined reference data through the `V2__seed_lookup_data.sql` migration script.

As this migration executes after `V1__core_schema.sql`, the lookup tables have already been created and are ready to accept inserted records. Each inserted row automatically generates a corresponding primary key value through MySQL's `AUTO_INCREMENT` mechanism, ensuring unique identifiers for all reference entries. These generated primary key values are referenced by foreign keys in related tables to maintain referential integrity. An example of seeding the `project_status` is shown below:

```
INSERT INTO project_status (name) VALUES
   ('IN_PROGRESS'),
   ('COMPLETED'),
   ('CANCELLED');
```

This query inserts predefined values into the `name` column within the `project_status` table, ensuring that required reference data is available immediately after migration.

2.8. Validate database integration

3. Core CRUD Functionality & User Interaction

3.1. Implement basic navigation and UI layout

The purpose of this task was to design and implement a basic navigation system and user interface layout for a Project Tracking System. The implementation was carried out using the Spring Boot framework following the Model-View-Controller (MVC) architectural pattern. The system enables users to navigate between core functional modules such as Projects, Timesheets, Expenses, and Reports through a consistent and responsive interface.

System Architecture

The implementation utilises several key technologies to support both backend functionality and frontend presentation. **Spring Boot** is used as the primary backend framework, providing a robust environment for handling HTTP requests, managing application configuration, and simplifying the development of controller classes. It enables rapid development through its convention-over-configuration approach.

Spring MVC is employed within the Spring Boot framework to manage the application's routing and request-handling logic. It facilitates the separation of concerns by directing

incoming requests to appropriate controller methods and returning corresponding views. [https://spring.io/projects/spring-boot]

For the presentation layer, **Thymeleaf** is used as a server-side template engine. It allows dynamic data to be embedded into HTML pages using expressions, ensuring seamless integration between the backend and frontend while maintaining clean and readable templates. [https://www.thymeleaf.org/]

On the frontend, **Bootstrap 5** is utilised to design a responsive and user-friendly interface. It provides pre-built components such as navigation bars, grids, buttons, and tables, enabling consistent styling and ensuring compatibility across different devices and screen sizes. [https://getbootstrap.com/docs/5.3/getting-started/introduction/]

Controller Design

The uiController class is responsible for handling HTTP GET requests and directing users to the appropriate views.
[https://medium.com/@aleksandr.shashnin/understanding-http-request-processing-in-spring-boot-2227ca339a9f]

```
@Controller
public class uiController {

   private final ProjectService projectService;

   public uiController(ProjectService projectService) {
       this.projectService = projectService;
   }

   @GetMapping({"/", "/dashboard"})
   public String dashboard(Model model) {

       long activeProjectCount = projectService.getActiveProjectCount();

       model.addAttribute("activeProjectCount", activeProjectCount);

       return "dashboard";
   }
```

The controller is annotated with @Controller, enabling Spring to recognise it as a component responsible for web request handling.

Dashboard Endpoint

The dashboard is mapped to both the root URL (/) and /dashboard. The controller retrieves the number of active projects from the service layer and passes this data to the view via the Model object.

This demonstrates:

- Integration between the controller and service layer [https://docs.spring.io/spring-framework/reference/web/webmvc.html]
- Dynamic data transfer to the frontend [https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/ui/Model.html]

Navigation Endpoints

Additional endpoints were implemented to support navigation:

- /search
- /timesheets
- /expenses/outlays
- /projects
- /employees
- /contacts
- /invoices
- /receipts
- /reports
- /users

Each endpoint returns a corresponding Thymeleaf template, enabling page routing without embedding logic in the view layer [https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html].

User Interface Design

The user interface is implemented using Thymeleaf templates combined with Bootstrap 5 for styling and responsiveness.

Layout Consistency

A consistent layout is achieved through the use of reusable components, specifically a navigation bar fragment. This ensures uniformity across all pages and reduces code duplication [https://www.thymeleaf.org/doc/articles/layouts.html].

Navigation Bar Implementation

The navigation bar is defined as a Thymeleaf fragment and included in multiple pages using the th:replace attribute [https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html#fragment-insertion].

The navbar provides links to all major system modules:

- Projects

- Contacts
- Employees
- Timesheets
- Expenses
- Invoices
- Receipts
- Reports

It also supports responsive behaviour via Bootstrap's navbar component [https://getbootstrap.com/docs/5.0/components/navbar/].

Role-Based Access Control

The navigation bar integrates with Spring Security to restrict access to certain features:

- The "Users" link is only visible to users with the ADMIN role
- This is achieved using the sec:authorize attribute [https://www.thymeleaf.org/doc/articles/springsecurity.html]

This enhances system security and ensures appropriate access control [https://docs.spring.io/spring-security/reference/index.html].

Dashboard Implementation

The dashboard serves as the main entry point of the application. Displaying the number of active projects retrieved from the backend, it provides quick access buttons to key modules.

Dynamic Rendering

Thymeleaf's iteration functionality (th:each) is used to dynamically generate table rows from backend data. This ensures that the UI reflects real-time application data [https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html#iteration].

Frontend Technologies

Bootstrap 5

Bootstrap is used to:

- Provide responsive layouts
- Standardise UI components such as buttons, tables, and navigation bars
- Ensure compatibility across different screen sizes

[https://getbootstrap.com/docs/5.0/getting-started/introduction/]

Thymeleaf

Thymeleaf enables:

- Server-side rendering of dynamic content
- Seamless integration with Spring MVC
- Clean separation of logic and presentation

[https://www.thymeleaf.org/documentation.html]

3.2. Implement CRUD for Projects

Overall Design

The Projects module was implemented as a full CRUD (Create, Read, Update, Delete) system using Spring Boot. A layered architecture was adopted consisting of Controller, Service, Repository, and DTO layers. This ensures consistency across the system while maintaining separation of concerns. [https://docs.spring.io/spring-boot/index.html]

The Projects module is inherently more complex and therefore does not extend the generic `BaseController` or `BaseService` classes used by other modules. While these base classes are effective for reducing duplication in straightforward CRUD scenarios, the Projects module involves substantially richer business logic, multiple interrelated entities, and integration across different modules. Because of this increased complexity, a more tailored and flexible architectural approach was necessary to properly encapsulate its behaviour and maintain clarity.

To support this, responsibilities are divided between multiple services:

- ProjectService - handles create, update, and delete operations
- ProjectQueryService - handles data retrieval, aggregation, and search
- ProjectFinanceService - handles financial calculations

This separation allows each concern to be managed independently, improving maintainability and scalability.

The Projects module acts as the central entity within the system, linking together data from Contacts, CostItems, Timesheets, Receipts, Invoices, and Project Reports. This makes it significantly more complex than previously implemented modules.

Create Functionality

The create functionality allows users to create a new project, capturing key information such as category, status, client, and start date.

The form is displayed using in the controller:

```
@GetMapping("/new")
public String newForm(Model model)
```

Dropdown data is populated dynamically:

```
projectQueryService.loadFormLookups(model);
```

This includes:

- Project categories
- Project statuses
- Contacts

Form submission is handled using:

```
@PostMapping
public String create(@Valid @ModelAttribute("projectForm") ProjectForm form,
                     BindingResult bindingResult,
                     Model model)
```

Validation is applied through annotations in the ProjectForm DTO, ensuring required fields such as category, client, and start date are provided.
[https://docs.spring.io/spring-framework/docs/3.0.x/reference/validation.html]

Within the service layer, the project is created:

```
Project project = new Project();
updateEntity(project, form);
```

Contacts are resolved by name rather than selected by ID:

```
contactRepository.findByNameIgnoreCase(form.getClientContactName())
```

This ensures that only existing contacts can be linked to a project, maintaining data consistency across modules.

Read Functionality

The read functionality supports both listing projects and retrieving detailed project information.

List View

Projects are retrieved using:

```
public List<ProjectView> list()
```

Each project is mapped to a ProjectView DTO, which presents key information such as title, status, category, and client name. This simplifies the data returned to the UI.

```
public Long getId() { return id; }
public String getTitle() { return title; }
public String getDescription() { return description; }
public String getStatus() { return status; }
public String getClientName(){return clientName;}
```

Detailed View

A more advanced read operation is implemented through:

```
public ProjectDetailsView getProjectDetails(Long id)
```

This method allows to view a specific project in more detail and aggregates data from multiple modules, including:

- CostItems (Outlays and Expenses)
- Timesheets (labour costs)
- Receipts
- Invoices

All data is combined into a single ProjectDetailsView object, which is passed to the UI. This reduces the need for multiple queries in the frontend and improves performance.

Service Layers

Due to the need for a detailed and aggregated project view, the service layer is divided into three specialised services.

Project Service
The ProjectService handles all write operations similar to all of the other modules' service layers (create, update, and delete) and ensures that project data is correctly validated and stored.

It maps data from the ProjectForm DTO into the Project entity, including category, status, start date, description, and associated contacts (client, solicitor, and insurance company).

```
project.setCategory(categoryRepository.findById(form.getCategoryId()).orElseThrow());
project.setStatus(statusRepository.findById(form.getStatusId()).orElseThrow());

Contact client = contactRepository.findByNameIgnoreCase(form.getClientContactName())
      .orElseThrow();
```

```
project.setClientContact(client);
```

This ensures that:

- Only valid categories and statuses are used
- Contacts must already exist in the system
- Data consistency is maintained across modules

It also enforces business rules during deletion by preventing removal of projects that have associated financial or operational data.

Project Query Service

The **ProjectQueryService** is responsible for retrieving and preparing project data for display.

It supports both listing and detailed views. For detailed views, it aggregates data from multiple sources including cost items, timesheets, receipts, and invoices.

```
BigDecimal outlayTotal = financeService.getOutlayTotal(id);
BigDecimal expenseTotal = financeService.getExpenseTotal(id);

BigDecimal totalExVat = outlayTotal.add(expenseTotal);
BigDecimal outstandingInvoices = financeService.getOutstandingInvoices(project);
```

This allows the system to present:

- Total expenses and outlays
- Outstanding invoice values

All data is combined into a ProjectDetailsView, reducing the need for multiple queries in the UI and improving performance.

Project Finance Service

The **ProjectFinanceService** handles all financial calculations related to a project.

It calculates totals for expenses, outlays, invoices and payments for each specific project ensuring consistent financial data across the system.

```
public BigDecimal getOutlayTotal(Long projectId) {
  return Optional.ofNullable(
        costItemRepository.sumByProjectAndType(projectId, CostItem.Type.OUTLAY)
  ).orElse(BigDecimal.ZERO);
}
```

It also calculates outstanding balances for each respective project:

```
public BigDecimal getOutstandingInvoices(Project project) {
  BigDecimal totalInvoiced = getTotalInvoiced(project);
  BigDecimal totalSettled = getEffectiveReceiptsTotal(project.getId());
  return totalInvoiced.subtract(totalSettled).max(BigDecimal.ZERO);
}
```

This ensures accurate tracking of:

- Project costs (expenses, outlays and timesheet entries)
- Outstanding invoice values

Update Functionality

The update functionality allows existing projects to be modified.

The edit form is loaded using:

```
@GetMapping("/{id}/edit")
```

Existing data is retrieved and mapped to a form:

```
projectService.getFormById(id);
```

Updates are processed using:

```
@PostMapping("/{id}")
```

The service layer updates the entity using a shared mapping method:

```
updateEntity(project, form);
```

This method handles:

- Updating category and status
- Resolving contact relationships
- Updating core project fields

Delete Functionality

The delete operation includes strict business rules to maintain system integrity.

Before deletion, the system checks for dependencies:

```
if (invoiceRepository.existsByProjectId(id))
if (costItemRepository.existsByProjectId(id))
if (timesheetRepository.existsByProjectId(id))
```

This prevents deletion of projects that are linked to financial or operational data.

Only projects with no dependencies can be deleted:

```
projectRepository.delete(project);
```

This ensures:

- Referential integrity
- Protection of financial data
- Prevention of orphaned records

Repository Layer

The repository layer is implemented using Spring Data JPA:

```
public interface ProjectRepository extends JpaRepository<Project, Long>,
JpaSpecificationExecutor<Project>
```

The use of JpaSpecificationExecutor enables dynamic querying for search functionality.[https://docs.spring.io/spring-data/jpa/reference/#specifications]

A custom method is defined to optimise data retrieval:

```
@EntityGraph(attributePaths = {"costItems"})
Optional<Project> findWithCostItemsById(Long id);
```

This improves performance by eagerly loading related data.

DTO Mapping and Data Transformation

Three DTOs are used:

ProjectForm

Used for input and validation. Contains:

- Category and status IDs
- Contact names
- Title, description, and start date

ProjectView

Used for list display. Contains:

- Basic project details
- Category and client information

ProjectDetailsView

Used for detailed display. Combines:

- Financial data
- Operational data
- Project documents

Mapping is handled in the service layer using methods such as:

```
mapToForm(Project project)
mapToView(Project project)
```

This ensures separation between the entity model and the UI.

The Projects module provides a comprehensive and centralised implementation of CRUD functionality. Unlike simpler modules, it integrates multiple data sources, supports financial aggregation, and includes advanced features such as dynamic search and aggregated views.

The decision to avoid generic base classes reflects the increased complexity of the module and allows greater flexibility in handling domain-specific logic. Through the use of multiple services, DTOs, and clearly defined business rules, the module remains maintainable while supporting complex system requirements.

3.3. Implement CRUD for Clients, Solicitors, Insurers & Suppliers

Overall Design

The Contacts module was implemented as a full CRUD (Create, Read, Update, Delete) system using Spring Boot. A layered architecture was adopted to separate concerns and improve maintainability. This consisted of:

- **Controller layer** – handles incoming HTTP requests and returns views
- **Service layer** – contains business logic and validation
- **Repository layer** – manages database access using Spring Data JPA
- **DTO (ContactForm)** – used for form binding and validation

[https://docs.spring.io/spring-boot/index.html]

For example, the controller is defined as:

```
@Controller
@RequestMapping("/contacts")
public class ContactController extends BaseController<Contact,
ContactForm>
```

This structure ensures the system is modular, reusable, and easier to extend across other parts of the system.

Create Functionality

The create functionality allows users to add new contacts through a web form. The form is displayed using:

```
@GetMapping("/new")
public String newForm(Model model) {
    model.addAttribute("contactForm", new ContactForm());
    model.addAttribute("mode", "create");
    return "contacts/form";
}
```

Form submission is handled by:

```
@PostMapping
public String create(@Valid @ModelAttribute("contactForm")
ContactForm form,
                     BindingResult bindingResult) {
```

When the form is submitted, Spring MVC automatically binds the input values to a populated `ContactForm` object using model binding. The request is handled by the controller method mapped with the `@PostMapping` annotation. Validation is applied through the `@Valid` annotation on the `ContactForm`, while any validation or binding errors are captured using the `BindingResult` object. If any errors are detected, the form is redisplayed with the previously entered data, allowing Thymeleaf to access the errors and present appropriate validation messages to the user within the interface.
[https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/validation/BindingResult.html]

Validation is applied using annotations in the DTO:

```
@NotBlank(message = "Name is required")
@Size(min = 2, max = 150, message = "Name must be between 2 and
150 characters")
private String name;
```

This ensures required fields are completed and data is within valid limits before saving.

The service layer processes the creation:

```
public Contact create(ContactForm form) {
    Contact contact = new Contact(form.getName());
    updateEntity(contact, form);
    return contactRepository.save(contact);
}
```

The reuse of the `updateEntity()` method avoids duplication and ensures consistency when mapping form data to the entity.

Read Functionality

The read functionality allows users to view all contacts or a single contact.

All contacts are retrieved using:

```
public List<Contact> list() {
    return contactRepository.findAll();
}
```

A single contact is retrieved using:

```
public Contact getById(Long id) {
    return contactRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Contact
not found"));
}
```

This ensures that invalid IDs are handled safely. On the frontend, Thymeleaf is used to display data dynamically: [https://www.thymeleaf.org/documentation.html]

```
<tr th:each="c : ${contacts}">
    <td th:text="${c.name}"></td>
</tr>
```

Update Functionality

The update functionality allows existing contacts to be edited. The edit form is pre-filled using:

```
@GetMapping("/{id}/edit")
public String editForm(@PathVariable Long id, Model model) {
    ContactForm form = contactService.getFormById(id);
    model.addAttribute("contactForm", form);
    model.addAttribute("mode", "edit");
    return "contacts/form";
}
```

Updates are processed using:

```
@PostMapping("/{id}")
public String update(@PathVariable Long id,
                     @Valid @ModelAttribute("contactForm")
ContactForm form,
                     BindingResult bindingResult) {
```

The service layer updates the entity:

```
public Contact update(Long id, ContactForm form) {
    Contact contact = getById(id);
    updateEntity(contact, form);
    return contactRepository.save(contact);
}
```

Mapping methods are used to convert between the entity and DTO:

```
public void updateEntity(Contact contact, ContactForm form) {
    contact.setName(form.getName());
    contact.setAddress(form.getAddress());
```

```
    contact.setPhone(form.getPhone());
    contact.setFax(form.getFax());
    contact.setComments(form.getComments());
}
```

This approach keeps the controller simple and maintains separation between layers.

Delete Functionality

The delete operation includes business rules to maintain data integrity:

```
public void delete(Long id) {
    Contact contact = getById(id);
```

Before deletion, the system checks if the contact is linked to other entities:

```
if (projectRepository.existsByClientContactId(id) ||
    projectRepository.existsBySolicitorContactId(id) ||
    projectRepository.existsByInsuranceCompanyContactId(id)) {
    throw new IllegalStateException("Contact is used in a
project.");
}
```

```
if (costItemRepository.existsBySupplierContact_Id(id)) {
    throw new IllegalStateException("Contact is used in a cost
item.");
}
```

Only if these checks pass is the contact deleted:

```
contactRepository.delete(contact);
```

This prevents invalid or orphaned data within the system.

Repository Layer

The repository layer is implemented using Spring Data JPA:

```
public interface ContactRepository extends JpaRepository<Contact,
Long>
```

This provides built-in CRUD operations without requiring manual SQL queries. Additional query methods are also defined:

```
Optional<Contact> findByName(String name);
Optional<Contact> findByNameIgnoreCase(String name);
```

These support the search functionality implemented at a later stage.
[https://docs.spring.io/spring-data/jpa/reference/]

Use of Base Classes

Generic base classes were used to reduce duplication:

```
public class ContactController extends BaseController<Contact,
ContactForm>
```

```
public class ContactService implements BaseService<Contact,
ContactForm>
```

This reduces duplication by allowing common CRUD logic to be reused across other modules such as projects and cost items.

Complex and Advanced Areas of the Contacts Module

While the Contacts module follows a clear CRUD structure, several areas of the implementation demonstrate more advanced design and logic beyond basic requirements.

1. Use of Generic Base Classes

One of the more complex aspects of the implementation is the use of generic base classes:

```
public class ContactController extends BaseController<Contact,
ContactForm>
```

```
public class ContactService implements BaseService<Contact,
ContactForm>
```

This introduces **generics**, which allow the same CRUD logic to be reused across multiple entities. Instead of rewriting similar controller and service logic for each module (e.g. Projects, Cost Items), the base classes provide common functionality that is extended. [https://www.geeksforgeeks.org/java/generic-class-in-java/]

2. DTO and Entity Mapping

Another complex area is the separation between the **DTO (ContactForm)** and the **entity (Contact)**, and the use of mapping methods:
[https://www.geeksforgeeks.org/java/data-transfer-object-dto-in-spring-mvc-with-example/]

```
public void updateEntity(Contact contact, ContactForm form) {
    contact.setName(form.getName());
    contact.setAddress(form.getAddress());
    contact.setPhone(form.getPhone());
    contact.setFax(form.getFax());
    contact.setComments(form.getComments());
}
```

```
public ContactForm mapToForm(Contact contact) {
    ContactForm form = new ContactForm();
    form.setId(contact.getId());
    form.setName(contact.getName());
    form.setAddress(contact.getAddress());
    form.setPhone(contact.getPhone());
    form.setFax(contact.getFax());
    form.setComments(contact.getComments());
    return form;
}
```

This improves data control and supports validation.

3. Validation Integration Across Layers

Validation is implemented at multiple level, from DTO annotations to frontend display using thymeleaf.
[https://medium.com/@epengfei/implementing-validation-on-spring-mvc-with-thymeleaf-129613626988]

```
<input th:field="*{name}"
       class="form-control"
       th:classappend="${#fields.hasErrors('name')} ?
```

```
'is-invalid'">

<div class="invalid-feedback"
     th:errors="*{name}"></div>
```

4. Dynamic Form Reuse

The same HTML form is reused for both creating and editing contacts. This is controlled using a mode variable:

```
<form th:action="${mode == 'edit'}
        ? @{/contacts/{id}(id=${contactForm.id})}
        : @{/contacts}"
      th:object="${contactForm}">
```

5. Business Rules in Delete Operation (Data Integrity)

The delete operation ensures that contacts cannot be removed if they are referenced elsewhere, demonstrating awareness of system relationships.

6. Optional Return URL Handling

The controller includes logic to optionally redirect users back to a previous page. For example, create contact can be accessed through the create project form. So once contact is created and saved it returnUrl returns it back to the projectform.
[https://www.baeldung.com/spring-redirect-and-forward?utm_source=chatgpt.com]

```
if (returnUrl != null && !returnUrl.isBlank()) {
    return "redirect:" + returnUrl;
}
```

This is used in both create and update operations and is passed through the form:

```
<input type="hidden" name="returnUrl" th:value="${returnUrl}" />
```

3.4. Implement CRUD for Employees

Overall Design

The Employees module was implemented using Spring Boot and follows a layered architecture consisting of Controller, Service, Repository, and DTO components. This ensures consistency with other modules in the system while maintaining clear separation of concerns.

However, unlike the other modules, the Employees module does not implement full CRUD functionality. Instead, it is designed as a **read-focused module** that integrates with the system's authentication layer. Employees are not managed independently; rather, they are linked to application users through the SystemUser entity.
[https://docs.spring.io/spring-security/reference/index.html]

This design ensures that only valid, active and authenticated users are associated with employee records. As a result, the module prioritises **data consistency and integration** over direct data manipulation.

The Employees module acts as a **supporting entity** within the system. It is used by other modules such as Timesheets and CostItems, where employee data is required for operational tracking and financial calculations.

Read Functionality

The Employees module supports listing and searching of employees.

List View

All employees are retrieved using:

```
@GetMapping
public String list(@RequestParam(value = "q", required = false)
String q, Model model)
```

If no search query is provided, all active employees are returned:

```
employees = employeeService.listSummaries();
```

If a query is provided, a filtered search is performed:

```
employees = employeeService.searchSummaries(query);
```

The results are added to the model and displayed in the UI. This approach improves usability, particularly when working with larger numbers of employees.
[https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html]

Search Functionality

Search is implemented in the service layer and supported by custom repository queries. Employees can be searched by:
[https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html]

- Username
- Employee name

Only **active users** are included in the results:

```
systemUserRepository.searchActiveUsers(query)
```

This ensures that inactive or disabled users are excluded from the system.

Service Layer

The service layer is responsible for retrieving and transforming employee data.

DTO Mapping

Employee data is converted into a simplified DTO:

```
.map(u -> new EmployeeView(
   u.getEmployee().getId(),
   u.getEmployee().getName()
))
```

The EmployeeView DTO contains only essential fields required for display:

- Employee ID
- Employee name

This reduces data transfer and avoids exposing unnecessary details from the underlying entity model.
[https://www.geeksforgeeks.org/java/data-transfer-object-dto-in-spring-mvc-with-example/]

Integration with SystemUser

A key feature of the Employees module is its integration with the authentication system.

The current employee is retrieved using:

```
public Employee getCurrentEmployee()
```

This method:

- Retrieves the logged-in user from the Spring Security context
- Finds the corresponding SystemUser
- Returns the associated Employee

[https://docs.spring.io/spring-security/reference/servlet/authentication/index.html]

```
Authentication authentication =
   SecurityContextHolder.getContext().getAuthentication();
```

```
String username = authentication.getName();

SystemUser user = systemUserRepository
   .findByUsername(username)
   .orElseThrow(() -> new RuntimeException("User not found"));

return user.getEmployee();
```

This design removes the need for manual employee selection and ensures that actions such as timesheet entries are always associated with the correct user.

Repository Layer

The repository layer is implemented using Spring Data JPA:
[https://docs.spring.io/spring-data/jpa/reference/repositories.html]

```
public interface SystemUserRepository extends
JpaRepository<SystemUser, Long>
```

Since employees are linked to system users, all queries are performed through the SystemUserRepository.

Custom queries are used to support search functionality:

```
@Query("""
select u
from SystemUser u
join u.employee e
where u.active = true
and (
   lower(u.username) like lower(concat('%', :q, '%'))
```

```
   or lower(e.name) like lower(concat('%', :q, '%'))
)
""")
List<SystemUser> searchActiveUsers(@Param("q") String q);
```

These queries:

- Join SystemUser with Employee
- Filter by active status
- Support case-insensitive searching

This improves usability while maintaining data integrity.
[https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html#jpa.query-methods.at-query]

DTO Design

The Employees module uses a lightweight DTO:

```
public class EmployeeView {
   private final Long id;
   private final String name;
}
```

This DTO is designed specifically for display purposes and avoids exposing the full Employee or SystemUser entities.
[https://www.geeksforgeeks.org/java/data-transfer-object-dto-in-spring-mvc-with-example/]

Design Decisions and Differences from Other Modules

Unlike other modules in the system, the Employees module does not implement full CRUD functionality. This is an intentional design decision based on how employee data is managed within the system.

- **No direct CRUD operations**: Employees are created and managed through the user (authentication) system rather than independently.
- **Integration with security**: Employee data is tightly coupled with SystemUser, ensuring that only authenticated users are considered valid employees.
- **Read-focused design**: The module focuses on retrieval and search functionality rather than data modification.

This differs from:

- **Contacts module**: Fully independent CRUD entity

- **CostItems module**: Complex financial and multi-entity relationships
- **Projects module**: Central aggregation and business logic
- **Timesheets module**: Operational tracking with financial impact

Complex and Advanced Areas of the Employees Module

1. Integration with Authentication System

The module is directly integrated with Spring Security, allowing the system to determine the current employee based on the logged-in user.
[https://docs.spring.io/spring-security/reference/servlet/authentication/index.html]

```
SecurityContextHolder.getContext().getAuthentication()
```

This reduces user input errors and improves consistency across modules.

2. Indirect Data Access via SystemUser

Instead of accessing Employee entities directly, all data is retrieved through the SystemUser entity:

```
u.getEmployee()
```

This introduces an additional abstraction layer but ensures that only valid and active users are considered.

3. Custom Search Queries Across Multiple Fields

Search functionality spans multiple attributes:

- Username
- Employee name

This is implemented using JPQL with joins and case-insensitive matching, improving usability. [https://docs.oracle.com/javaee/7/tutorial/persistence-querylanguage.htm]

4. Use of DTO for Data Simplification

The use of EmployeeView ensures that only required data is returned to the UI, maintaining separation between layers and improving performance.

5. Role as a Supporting Module

The Employees module is not standalone. It is used by:

- Timesheets (to record work carried out by employees)
- CostItems (to associate costs with employees)

This makes it a critical part of the system despite its simpler functionality.

The Employees module provides a streamlined and integrated approach to managing employee data within the system. Rather than implementing full CRUD functionality, it focuses on retrieving and managing employees through their association with system users.

By integrating with the authentication layer, the module ensures that only active and valid users are treated as employees. The use of DTOs, custom queries, and service-layer mapping maintains a clean and maintainable architecture.

Although less complex than modules such as Projects or CostItems, the Employees module plays a crucial role in supporting system functionality, particularly in areas such as timesheet tracking and financial calculations.

3.5. Implement CRUD for Timesheets

Overall Design

The Timesheets module was implemented using the same layered architecture as the previous modules (Contacts and CostItems), consisting of Controller, Service, Repository, and DTO layers. This ensures consistency across the system while reducing duplication through the reuse of generic base classes.
[https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html#using.structuring-your-code]

However, the Timesheets module introduces functionality specifically designed to track work carried out by employees on projects. Each timesheet entry is linked to a project and records details such as date, work description, hours worked, and a calculated charge. Unlike simple data storage, this module also performs a key role in supporting invoice generation. The recorded hours are converted into a monetary value based on the employee's hourly rate, allowing labour costs to be calculated and aggregated per project. These values can then be included in invoices, meaning timesheets contribute directly to the financial output of the system.

This also introduces additional considerations compared to earlier modules. Timesheet entries can be linked to an invoice once billed, and as a result, certain operations such as deletion are restricted to maintain financial integrity. This shifts the module beyond basic CRUD functionality into state-aware behaviour, similar to the CostItems module, where data must remain consistent once used in invoicing.

Unlike the Contacts module, which manages largely standalone data, and the CostItems module, which focuses on explicit financial transactions, the Timesheets module focuses on

recording employee activity while also bridging the gap between operational work and financial processing.

Create Functionality

The create functionality allows employees to record work completed on a project.

Form Display:

```
@GetMapping("/new")
public String createForm(Model model)
```

The controller prepares the form and loads required dropdown data:

```
model.addAllAttributes(timesheetEntryService.getFormLookups());
```

This dynamically provides:

- Projects
- Work descriptions (from a predefined list)

This follows Spring MVC model handling practices [https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html]

Form Submission:

```
@PostMapping
public String create(@Valid @ModelAttribute("form")
TimesheetEntryForm form,
                     BindingResult result,
                     Model model)
```

Validation is applied using annotations in the DTO: [[https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/validation/BindingResult.html]

```
@NotNull(message = "Project is required")
private Long projectId;

@NotNull(message = "Entry date is required")
private LocalDate entryDate;

@PositiveOrZero(message = "Hours must be zero or positive")
```

```
private BigDecimal hours;
```

Service Layer Logic:

```
Project project =
projectRepo.findById(form.getProjectId()).orElseThrow();
Employee employee = currentUserService.getCurrentEmployee();
```

One notable design decision is that the employee is automatically assigned based on the currently logged-in user, rather than being manually selected. This enhances usability and ensures data consistency.
[https://docs.spring.io/spring-security/reference/servlet/authentication/index.html]

The work description is resolved using:

```
WorkDescription workDesc = resolveWorkDescription(form);
```

This includes support for an **"Other" option**, allowing users to enter a custom description if it does not already exist.

The entity is then created and saved:

```
TimesheetEntry entry = new TimesheetEntry(
   project,
   employee,
   workDesc,
   form.getEntryDate(),
   form.getHours()
);
```

Read Functionality

The read functionality allows users to view timesheet entries, either as a full list or filtered by project.

All entries are retrieved using:

```
public List<TimesheetEntryView> list()
```

Each entity is converted into a **view-specific DTO**:

```
.map(this::toView)
```

The TimesheetEntryView combines data from multiple related entities:

- Project (title)
- Employee (name)
- Work description
- Calculated charge
- Billing status

This improves UI clarity and avoids exposing the internal entity model.
[https://www.geeksforgeeks.org/java/data-transfer-object-dto-in-spring-mvc-with-example/]

Project-Based Filtering:

```
public List<TimesheetEntryView> findByProjectId(Long projectId)
```

This ensures that all work carried out on a project can be easily accessed when generating invoices.

Update Functionality

The update functionality allows existing timesheet entries to be modified.

Load Edit Form:

```
@GetMapping("/{id}/edit")
```

This retrieves the existing data and maps it to a form object:

```
TimesheetEntryForm form = timesheetEntryService.getFormById(id);
```

Process Update:

```
@PostMapping("/{id}")
```

In the service layer, the entity is updated:

```
entry.setProject(project);
entry.setEmployee(employee);
entry.setWorkDescription(workDesc);
entry.setEntryDate(form.getEntryDate());
entry.setHours(form.getHours());
```

As with creation, the employee is derived from the current user, ensuring consistency.

Delete Functionality

The delete operation includes a business rule to maintain financial integrity:

```
if (entry.getInvoice() != null) {
   throw new IllegalStateException("Cannot delete a billed
timesheet entry.");
}
```

This ensures that:

- Timesheet entries already included in an invoice cannot be removed
- Financial records remain accurate and consistent

Only unbilled entries can be safely deleted:

```
repository.delete(entry);
```

Repository Layer

The repository extends Spring Data JPA:
[https://docs.spring.io/spring-data/jpa/reference/repositories.html]

```
public interface TimesheetEntryRepository extends
JpaRepository<TimesheetEntry, Long>
```

Custom query methods are used to support business requirements:

Project-based filtering

```
List<TimesheetEntry> findByProject_Id(Long projectId);
```

This method retrieves all timesheet entries associated with a specific project. By using `Project_Id`, Spring Data JPA automatically derives a query based on the foreign key relationship, eliminating the need for a manually defined query.

This functionality enables users to view all work completed for a particular project, supports project-level reporting, and facilitates the aggregation of data required for invoice generation.

Invoice Filtering

```
List<TimesheetEntry> findByProjectAndInvoiceIsNull(Project
project);
```

This method returns all timesheet entries for a given project that have not yet been assigned to an invoice. The condition `InvoiceIsNull` ensures that only unbilled work is retrieved.

This is important for identifying billable work that has not yet been invoiced, preventing duplicate billing of the same timesheet entries and ensuring accurate and controlled invoice creation .

Aggregation for invoicing

```
@Query("""
SELECT COALESCE(SUM(t.hours * e.hourlyRate),0)
FROM TimesheetEntry t
JOIN t.employee e
WHERE t.project.id = :projectId
""")
```

This calculates the total labour cost for a project by multiplying hours worked by employee hourly rates. Performing this calculation at database level improves performance and supports invoice generation.

The use of `COALESCE` ensures that a value of `0` is returned when no matching records exist, preventing null-related errors in further processing.
[https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html]

DTO Mapping and Data Transformation

Two DTOs are used:

TimesheetEntryForm

Used for input and validation. Contains:

- Project ID
- Work description
- Date
- Hours

TimesheetEntryView

Used for display. Combines:

- Project title
- Employee name
- Work description
- Charge (calculated)
- Billing status

Mapping is handled in the service layer:

```
private TimesheetEntryView toView(TimesheetEntry entry)
```

The charge is calculated using:

```
entry.getNetAmount().setScale(2, RoundingMode.HALF_UP);
```

This ensures financial values are correctly formatted.
[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/RoundingMode.html]

Complex and Advanced Areas of the Timesheets Module

1. Automatic Employee Assignment

The system automatically assigns the currently logged-in employee:

```
currentUserService.getCurrentEmployee();
```

This removes the need for manual selection and ensures accuracy.
[https://docs.spring.io/spring-security/reference/index.html]

2. Dynamic Work Description Handling

The system supports both predefined and custom work descriptions:

```
if ("Other".equalsIgnoreCase(selected.getName()))
```

Selecting “Other” allows users to create a new description and store it in the database, balancing flexibility with structured data management.

3. Multi-Entity Relationships

Each timesheet entry is linked to:

- Project
- Employee
- Work Description
- Invoice (optional)

These relationships increase complexity and require careful validation and mapping. [https://docs.oracle.com/javaee/7/tutorial/persistence-intro.htm]

4. Charge Calculation and Financial Integration

Charges are calculated based on:

- Hours worked
- Employee hourly rate

This directly supports invoice generation and integrates the Timesheets module with the financial aspects of the system.

5. Business Rules for Billing Integrity

Consistent with the CostItems module, timesheet entries cannot be modified or deleted once billed. This ensures:

- Accurate financial reporting
- Protection of invoiced data

The Timesheets module extends the core CRUD functionality by introducing employee work tracking linked to projects. It plays a key role in supporting invoice generation by recording hours worked and calculating associated charges.

While less complex than the CostItems module, it introduces important features such as automatic employee assignment, dynamic work descriptions, and financial integration. The use of DTOs, layered architecture, and business rules ensures the module is robust, maintainable, and aligned with the overall system design.

3.6. CRUD for Outlays and Expenses

Overall Design

The CostItems module is implemented as a full CRUD system using Spring Boot and follows a layered architecture consisting of Controller, Service, Repository, and DTO layers. [https://docs.spring.io/spring-boot/redirect.html?page=using#using.structuring-your-code].
This consists of:

- **Controller layer** – handles HTTP requests and returns views
- **Service layer** – contains business logic and validation

- **Repository layer** – manages database access using Spring Data JPA
- **DTOs (CostItemForm, CostItemView)** – used for form binding and display

The controller extends a generic base class:
[https://www.geeksforgeeks.org/java/generic-class-in-java/]

```
public class CostItemController extends BaseController<CostItem,
CostItemForm>
```

This allows reuse of common CRUD functionality and ensures consistency across modules such as Contacts and Projects.

Compared to simpler modules such as Contacts, the CostItems module introduces additional complexity due to:

- Handling two types: **Outlays and Expenses**
- Managing relationships multiple entities (Project, Employee, Supplier)
- Processing financial data (costs)
- Integration with invoicing (billed vs unbilled)

These characteristics align with enterprise application patterns involving transactional data and domain relations. [Fowler, M., Rice, D., Foemmel, M., Hieatt, E., Mee, R., & Stafford, R. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. ISBN: 0-321-12742-0. https://martinfowler.com/books/eaa.html]

Create Functionality

The create functionality allows users to add new cost items (either Outlays or Expenses) through a web form.
[https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html]

Form Display:

```
@GetMapping("/new")
public String createForm(Model model)
```

Dropdown data is populated dynamically:

```
model.addAllAttributes(costItemService.getDropdowns());
```

This includes:

- Projects
- Employees
- Contacts (for suppliers)

- Cost item types

Form Submission:

```
@PostMapping
public String saveCostItem(@Valid @ModelAttribute("costItemForm")
CostItemForm form,
                           BindingResult result,
                           Model model)
```

Validation is applied using annotations in the DTO:
[[https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/validation/BindingResult.html]

```
@NotNull(message = "Cost amount is required")
@PositiveOrZero(message = "Cost amount must be zero or positive")
private BigDecimal costAmount;
```

A key aspect of CostItems module is the handling of **Outlays vs Expenses**:

```
if (type == CostItem.Type.OUTLAY) {
   if (form.getSupplierContactId() == null) {
       throw new IllegalStateException("Outlays must be linked to
a supplier.");
   }
}
```

This introduces our business rules ensuring outlays must have a supplier and expenses do not require one.

Read Functionality

The read functionality allows users to view all cost items or individual records.

All cost items are retrieved using a view specific DTO:

```
public List<CostItemView> listViews()
```

Unlike the Contacts module, a dedicated **CostItemView** DTO is required due to the complexity of the data. CostItems combine information from multiple related entities, including Projects, Employees, and optional Supplier Contacts, as well as computed information such as billing status. The DTO presents this data in a simplified, user-friendly format while avoiding direct exposure of the entity model.
[https://www.geeksforgeeks.org/java/data-transfer-object-dto-in-spring-mvc-with-example/]

```
return costItemRepository.findAllWithDetails()
        .stream()
        .map(this::toView)
        .toList();
```

Update Functionality

The update functionality allows existing cost items to be edited.

The edit form is loaded using:

```
@GetMapping("/{id}/edit")
```

The update is processed through the same method used for creation:

```
@PostMapping
```

The service layer enforces a key business rule:

```
if (!item.isUnbilled()) {
   throw new IllegalStateException("Billed cost items cannot be
edited.");
}
```

This re-iterates our business rule that once a cost item is included in an invoice, it cannot be modified, maintaining financial accuracy and data integrity.

Delete Functionality

The delete operation also includes business rules to prevent invalid operations.

```
public void delete(Long id)
```

Before deletion, the system checks:

```
if (!item.isUnbilled()) {
   throw new IllegalStateException("Billed cost items cannot be
deleted.");
}
```

This prevents:

- Deletion of invoiced items
- Breaking financial records

Only unbilled cost items can be safely deleted.

Repository Layer

The repository layer is implemented using Spring Data JPA: [https://docs.spring.io/spring-data/jpa/reference/repositories.html]

```
public interface CostItemRepository extends
JpaRepository<CostItem, Long>
```

In addition to standard CRUD operations, custom queries were implemented to support business requirements.

Filtering

```
List<CostItem> findByProjectAndType(Project project, CostItem.Type
type);
```

This allows retrieval of cost items by project and type (Outlay or Expense), supporting structured reporting. This is important as Projects may contain many cost items and users often need to view only **expenses or only outlays.** This supports clearer reporting and aligns with the requirement to organise costs within projects.
[https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html]

Aggregation (important for invoicing)

```
@Query("""
SELECT COALESCE(SUM(c.costAmount),0)
FROM CostItem c
WHERE c.project.id = :projectId
AND c.type = :type
""")
```

This query calculates total expenses and total outlays per project. These values are essential for invoice generation, financial reporting, and project cost tracking. Performing this calculation at the database level is more efficient than processing it in application code.
[https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html#jpa.query-methods.at-query]

- SUM(c.costAmount) calculates the total cost
- COALESCE(..., 0) ensures that if no records exist, the result is **0 instead of null**
- :projectId and :type are parameters passed into the query

Fetch optimisation

```
@Query("""
SELECT c FROM CostItem c
JOIN FETCH c.project
JOIN FETCH c.employee
LEFT JOIN FETCH c.supplierContact
""")
```

This query makes use of **`JOIN FETCH`**, which is an important performance optimisation technique in JPA. By default, JPA applies lazy loading to relationships, meaning that related entities such as Project, Employee, and Supplier are retrieved separately when accessed. This can lead to the **N+1 query problem**, where multiple unnecessary database queries are executed, negatively impacting performance.

To address this, `JOIN FETCH` is used to retrieve related entities as part of a single query. [https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html#jpa.query-methods.at-query] For example:

- `JOIN FETCH c.project` retrieves the associated project data
- `JOIN FETCH c.employee` retrieves the associated employee data
- `LEFT JOIN FETCH c.supplierContact` retrieves the supplier data if it exists

By loading all required data in one query, this approach significantly reduces the number of database calls. As a result, it improves overall system performance, particularly when displaying lists of cost items, leading to faster page loading and more efficient data retrieval.

DTO Mapping and Data Transformation

Data transfer between the user interface and the database layer is managed through the use of Data Transfer Objects (DTOs). This approach separates the internal entity model from the data exposed to the user, improving maintainability and security.

Mapping between the form and the entity is handled through a method in the service layer:

```
public void updateEntity(CostItem entity, CostItemForm form)
```

This method is responsible for:

- Loading related entities such as Project, Employee, and Supplier from their respective repositories
- Applying business rules, including validation specific to Outlays and Expenses
- Setting all relevant fields on the CostItem entity

Mapping to Form:

```
public CostItemForm mapToForm(CostItem c)
```

This method converts an existing entity into a form object, allowing it to be displayed and edited in the user interface.

In addition, a separate **CostItemView DTO** is used for presenting data in the UI:

```
private CostItemView toView(CostItem item)
```

The `CostItemView` DTO is designed specifically for display purposes and combines data from multiple related entities into a single, user-friendly structure.

This separation of DTOs and entities ensures:

- A clean and well-structured architecture
- Secure handling of input data by avoiding direct exposure of entities
- Flexible and efficient UI rendering, particularly when displaying combined or computed data

Complex and Advanced Areas of the CostItems Module

1. Handling Multiple Types (Outlays vs Expenses)

The module supports both Outlays and Expenses within a single entity, requiring conditional validation logic. This introduces complexity beyond standard CRUD operations.

2. Multi-Entity Relationships

Each cost item is linked to:

- A Project
- An Employee
- An optional Supplier (Contact)

These relationships increase complexity due to multiple dependencies and validation requirements.

```
Project project =
projectRepository.findById(form.getProjectId()).orElseThrow();
Employee employee =
employeeRepository.findById(form.getEmployeeId()).orElseThrow();
```

```
entity.setProject(project);
entity.setEmployee(employee);
entity.setSupplierContact(supplierContact);
```

This increases complexity due to:

- Multiple foreign key relationships
- Dependency on other modules
- Additional validation requirements

3. Financial Data and Aggregation

The use of `BigDecimal` for cost values and database-level aggregation queries enables accurate financial calculations required for invoicing and reporting.
[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigDecimal.html]

4. Business Rules for Billing Integrity

Strict rules prevent modification or deletion of billed items, ensuring consistency and accuracy in financial data.

5. Use of View DTOs
The `CostItemView` DTO combines data from multiple entities and includes computed fields such as billing status, improving data presentation while maintaining separation from the entity model.
[https://www.geeksforgeeks.org/java/data-transfer-object-dto-in-spring-mvc-with-example/]

6. Dynamic Form Data (Dropdowns)

Dropdown data is dynamically loaded from multiple repositories:
[https://medium.com/@aravindcsebe/from-database-to-dropdown-a-full-stack-journey-in-real-time-data-loading-c92b3f772622]

```
public Map<String, Object> getDropdowns() {
   Map<String, Object> data = new HashMap<>();
```

```
  data.put("projects", projectRepository.findAll());
   data.put("employees", employeeRepository.findAll());
   data.put("contacts", contactRepository.findAll());
   data.put("types", CostItem.Type.values());
   return data;
```

```
}
```

This improves usability while increasing backend complexity. However, it also adds backend complexity as multiple repositories must be accessed.

7. Integration with Invoicing System

Cost items are linked to invoices and tracked using billing status:

```
(c.invoice IS NOT NULL)
```

This is reflected in the view DTO:

```
private boolean billed;
```

And enforced in business logic:

```
item.getInvoice() != null
```

This enables:

- Tracking billed vs unbilled items
- Preventing invalid edits/deletes
- Supporting invoice generation

Differences Between Contacts and CostItems Modules (Enhanced with Code)

While both modules follow a similar CRUD and layered architecture, the CostItems module is significantly more complex.

- **Business Logic:** Contacts have minimal validation, while CostItems include conditional and state-based rules
- **Relationships:** Contacts are largely standalone, whereas CostItems manage multiple entity relationships
- **Financial Handling:** CostItems support aggregation and reporting, which Contacts do not
- **DTO Usage:** Contacts use a single DTO, while CostItems use separate form and view DTOs
- **CRUD Behaviour:** CostItems implement state-aware CRUD (based on billing status)

The CostItems module extends the basic CRUD pattern by incorporating advanced business logic, multi-entity relationships, and financial data processing. It directly satisfies the project requirement to manage outlays and expenses within projects and supports accurate invoice generation.